



Universidad de
los Andes

FACULTAD DE
INGENIERÍA Y CIENCIAS
APLICADAS

INFORME PROYECTO 2

Juego de Reflejos en Assembly sobre Pochoco SoC

Instanciación del Pochoco SoC en FPGA e implementación del juego en assembly
Lattice iCE40 HX1K — Nandland Go Board

Integrantes: Lucas Nestler, Santiago Arnolds, Matías Veto
Curso: Arquitectura de Computadores, semestre 2026-2
Profesor: Jorge Gómez Mir

Entrega en GitHub: 29 de septiembre · Demostración en clases: 30 de septiembre

Índice

1. Arquitectura del juego, contador de ciclos y temporización de 3 s	3
1.1 Diagrama de bloques	3
1.2 Mapa de memoria usado por el juego	3
1.3 Contador de ciclos	4
1.4 Frecuencia de reloj y conversión ciclos → tiempo	4
1.5 Secuencia de una ronda (arquitectura del juego)	4
1.6 Generación pseudoaleatoria	5
2. Programa en assembly: medición de tiempo y promedio de diez rondas	6
2.1 Convención de registros	6
2.2 Medición del tiempo de respuesta	6
2.3 Conversión a décimas de segundo y cálculo del promedio	6
2.4 Despliegue en los 7 segmentos sin división/multiplicación	7
2.5 Manejo de pulsación incorrecta	7
3. Assembler propio: codificación, etiquetas, validación y pruebas	8
3.1 Sintaxis aceptada	8
3.2 Instrucciones soportadas (usadas por el juego)	8
3.3 Tratamiento de etiquetas	9
3.4 Validación y mensajes de error	9
3.5 Ejemplo de traducción: assembly → código máquina	9
3.6 Pruebas	10

1. Arquitectura del juego, contador de ciclos y temporización de 3 s

El punto de partida es el repositorio base pochoco_soc: el Espino Core (pipeline RV32E: fetch, decode, ALU, banco de registros, load/store y controlador) conectado en Pochoco SoC a una RAM unificada de instrucciones/datos y a periféricos mapeados en memoria. Sobre esa base se agregó un módulo superior propio, rtl/game_top.v, que instancia pochoco_soc seleccionando sw/game.hex como imagen de memoria y conectando directamente los pines físicos de la Go Board (LEDs, switches, displays de 7 segmentos y SPI) según goboard.pcf. No se agregó lógica adicional en game_top.v a propósito: toda la lógica del juego vive en el software (sw/game.s), que corre sobre el mismo SoC sin modificaciones al camino de datos del núcleo.

1.1 Diagrama de bloques

El siguiente diagrama resume cómo game_top se apoya en pochoco_soc y en los periféricos relevantes para el juego.

```
+-----+
| game_top.v (FPGA) |
| |
| i_Clk (25 MHz) ---+ |
| v |
| +-----+ pochoco_soc -----+
| |
| | +-----+ instr +-----+ data +-----+ | | | | | | |
| | | Espino Core|<----->| pochoco_ram |<----->| decoder| |
| | | (RV32E) | | (sw/game.hex) | |(addr[31|
| | +-----+ +-----+ | ,16]) | |
| | +---+---+ |
| | bus 0x8000_0000 | |
| | +-----+ |
| | v |
| | +-----+ +-----+ |
| | | pochoco_periph | | pochoco_spi_slave| |
| | | 0x00 7-seg | | 0x8001_0000 | |
| | | 0x04 LEDs | | (no usado por | |
| | | 0x08 botones | | el juego) | |
| | | 0x0C contador | +-----+ |
| | | de ciclos | |
| | +---+---+---+---+ |
| | +-----+|---+|---+|-----+
| | |
| | o_LED[3:0] | i_Switch[3:0] (botones, via debounce)
| | o_Segment1/2 (7-seg activo bajo)
+-----+
```

1.2 Mapa de memoria usado por el juego

Dirección / offset	Registro	Acceso	Descripción
0x0000_0000	RAM unificada	R/W	Instrucciones y datos (game.hex se carga aquí al boot).
0x8000_0000 + 0x00	Displays 7-seg	W	Byte con dos nibbles hex, decodificados por hex2seg (activo bajo en la placa).
0x8000_0000 + 0x04	LEDs	W	4 bits, uno por LED (one-hot para el LED objetivo, 0xF para los 4 encendidos).
0x8000_0000 + 0x08	Botones	R	4 bits ya debounced (debounce_bank), botón i asociado al LED i.

Dirección / offset	Registro	Acceso	Descripción
0x8000_0000 + 0x0C	Contador de ciclos	R/W	32 bits, incrementa en cada ciclo de reloj desde reset. Escribir un valor recarga el contador a ese valor (0 lo reinicia). Es la única fuente de tiempo del juego.

1.3 Contador de ciclos

El contador de ciclos vive en rtl/pochoco_periph.v (offset 0x0C), como un registro de 32 bits que se incrementa en cada flanco de reloj desde el reset del SoC. Al no requerir ninguna instrucción de temporizador dedicada, el programa lo usa como única fuente de tiempo: leerlo entrega el conteo actual, y escribirle cualquier valor lo recarga a ese valor (el programa nunca escribe al offset 0x0C: para medir un intervalo guarda el valor leído al comienzo en un registro y resta el valor leído al final, sin reiniciar el contador global y sin perder continuidad de tiempo entre fases).

```
always @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) cyc_q <= 32'b0;
  else if (access & we_i & (off == 6'd3)) cyc_q <= wdata_i; // escritura recarga
  else cyc_q <= cyc_q + 32'd1; // incrementa cada ciclo
end
```

1.4 Frecuencia de reloj y conversión ciclos → tiempo

El reloj de la Go Board es de 25 MHz (mismo oscilador del Proyecto 1, goboard.pcf), es decir 25 000 000 ciclos por segundo. De ahí las dos constantes de temporización que usa game.s:

Magnitud	Cálculo	Ciclos	Uso en el programa
3 segundos	$3 \text{ s} \times 25\,000\,000 \text{ ciclos/s}$	75 000 000	Espera de los 4 LEDs encendidos al inicio de cada ronda (wait_3s).
1 décima de segundo	$0.1 \text{ s} \times 25\,000\,000 \text{ ciclos/s}$	2 500 000	Unidad usada para convertir el tiempo de reacción y el promedio (response_div / average_div).

Como la ALU del Espino Core no implementa multiplicación/división (ver §2.4), ambas constantes se cargan como inmediatos de 32 bits partidos en lui + addi (por ejemplo, 75 000 000 = 0x0479_5E00 se arma con lui t1, 0x4787 seguido de addi t1, t1, -1856, porque el campo inmediato de addi es de 12 bits con signo y el nibble bajo de 75 000 000 excede ese rango positivo, así que se resta el complemento).

1.5 Secuencia de una ronda (arquitectura del juego)

- Inicio de ronda / medición de tiempo: se lee el contador de ciclos en cyc_start antes de encender ningún LED — éste es el mismo instante lógico en que arranca la ronda, tal como exige el enunciado, y no el instante en que se enciende el LED objetivo.
- Fase de 3 s: se escriben los 4 bits de LED en 1 (0xF) y se hace polling del contador de ciclos hasta que ciclos_transcurridos $\geq$ 75 000 000.
- Selección pseudoaleatoria: terminada la espera, se calcula el índice 0..3 del LED objetivo (ver §1.6) y se escribe la máscara one-hot correspondiente en el registro de LEDs, apagando los otros tres.

- Espera de pulsación: primero se espera a que el jugador suelte cualquier botón que hubiese quedado presionado de la ronda anterior (evita falsos aciertos instantáneos), luego se hace polling del registro de botones hasta ver una pulsación distinta de 0.
- Verificación: se relee el contador de ciclos y se resta `cyc_start` para obtener el tiempo de reacción en ciclos. Si el botón presionado coincide con la máscara objetivo, la ronda es correcta; si no, se muestra "EE" en los displays, se apagan los LEDs, se espera una pausa visible de error y se reinicia solo la ronda actual (el contador de rondas correctas y la suma de tiempos acumulados no se tocan).
- Cierre de ronda correcta: el tiempo de reacción se convierte a décimas de segundo (§1.4), se acumula en el promedio y se muestra en los displays; al llegar a 10 rondas correctas el programa calcula y muestra el promedio y entra en un bucle infinito (halt) dejando el resultado visible.

1.6 Generación pseudoaleatoria

El enunciado permite usar "una semilla tomada del contador de ciclos y operaciones de assembly". Como la ALU tiene los corrimientos (SLL/SRL/SRA/SLLI/SRLI/SRAI) deshabilitados en hardware (ejecutan como ADD, ver §2.4), no es viable un LFSR clásico basado en shifts. En su lugar, `s1` se mantiene como un estado persistente entre rondas que se mezcla cada ronda con: (a) el jitter humano de la espera de 3 s (`t2`, los ciclos que sobraron sobre el umbral), (b) una constante XOR fija para romper simetrías, y (c) el instante de inicio de la ronda (`cyc_start`). El índice de LED objetivo se obtiene con `and i a3, s1, 3` (1 de 4, sin necesitar shift porque 4 es potencia de 2). La semilla inicial al bootear es el valor del contador de ciclos en ese instante; desde la ronda 2 en adelante la secuencia depende del tiempo de reacción real del jugador, que varía entre partidas y entre rondas, cumpliendo el requisito de que la selección "no puede ser una secuencia fija que se repita exactamente en cada partida". El testbench de extremo a extremo (`sim/tb_game_smoke.v`) valida explícitamente que se observe más de un LED objetivo distinto en una misma partida simulada.

Nota de diseño: la primera ronda de cada partida siempre parte de una semilla muy similar (el conteo de ciclos entre el reset y la instrucción `lw s1, 12(s0)` en `_start` es casi constante), por lo que el primer LED objetivo tiende a repetirse entre encendidos de la placa. A partir de la segunda ronda la mezcla con el tiempo de reacción humano rompe ese patrón. No afecta el cumplimiento del requisito (que exige variedad perceptible dentro de la partida, no independencia estadística estricta), pero se documenta aquí como una limitación conocida.

2. Programa en assembly: medición de tiempo y promedio de diez rondas

El programa completo vive en `sw/game.s` (204 líneas, RV32E, sin corrimientos ni multiplicación/división). No usa pila ni llamadas anidadas: una única subrutina, `show_decimal`, se invoca con `jal ra, ...` y retorna con `jalr x0, 0(ra)`.

2.1 Convención de registros

Registro	Uso
s0	Base de periféricos, fija en 0x8000_0000 durante todo el programa.
s1	Estado persistente del generador pseudoaleatorio (sobrevive entre rondas).
a0	correct_count: rondas correctas acumuladas (0..10).
a1	sum_decimas: acumulador de décimas de segundo para el promedio.
a2	cyc_start: contador de ciclos al iniciar la ronda actual.
a3 / a4	Índice (0..3) y máscara one-hot del LED objetivo de la ronda actual.
a5, t0-t2, ra	Registros de trabajo ("scratch"), reutilizados libremente entre fases.

2.2 Medición del tiempo de respuesta

El tiempo de reacción se mide íntegramente en hardware/software combinados: a2 guarda el valor del contador de ciclos leído al comienzo de la ronda (antes de encender los 4 LEDs, ver §1.5); cuando el jugador presiona un botón se vuelve a leer el contador y se resta a2, obteniendo directamente los ciclos transcurridos desde el inicio de la ronda — es decir, incluye los 3 s de espera inicial más el tiempo real de reacción tras encenderse el LED objetivo. Esto es intencional: el enunciado pide que "la medición de tiempo arranca en el mismo instante lógico en que comienza la ronda", así que el tiempo mostrado siempre incluye la ventana fija de 3 s. Extracto relevante:

```
round_start:
sw x0, 0(s0) # limpia el display
lw a2, 12(s0) # a2 = cyc_start (arranca el cronometro AQUI)
addi t0, x0, 15
sw t0, 4(s0) # LEDs = 1111 (fase de 3 s)
...
wait_button:
lw t0, 8(s0)
beq t0, x0, wait_button
lw t1, 12(s0)
sub t1, t1, a2 # t1 = ciclos de reaccion (incluye los 3 s)
```

2.3 Conversión a décimas de segundo y cálculo del promedio

Sin instrucción de división en la ALU, la conversión ciclos → décimas de segundo se calcula por resta repetida: se resta 2 500 000 (1 décima a 25 MHz) del total de ciclos tantas veces como sea posible, contando las restas en a5. El mismo patrón se reutiliza para el promedio final (dividir sum_decimas entre 10) y, dentro de `show_decimal`, para separar decenas y unidades del valor a mostrar.

```
# decimas de segundo = t1 / 2,500,000 (resta repetida)
lui t2, 0x262
addi t2, t2, 1440 # t2 = 2,500,000 ciclos (1 decima a 25 MHz)
addi a5, x0, 0
response_div:
blt t1, t2, response_div_done
```

```

sub t1, t1, t2
addi a5, a5, 1
jal x0, response_div
response_div_done:
add a1, a1, a5 # sum_decimas += decimas de esta ronda
addi a0, a0, 1 # correct_count += 1

```

Al completar la décima ronda correcta ($a0 == 10$), el programa salta a `all_rounds_done` y aplica la misma resta repetida para dividir `a1` (suma de las 10 décimas) entre 10, obteniendo el promedio en la misma unidad (décimas de segundo), que se muestra con `show_decimal` y queda fijo en pantalla porque el programa termina en un bucle infinito (`halt: jal x0, halt`).

2.4 Despliegue en los 7 segmentos sin división/multiplicación

`show_decimal` recibe en `a5` un valor de 0 a 99 (se recorta a 99 si se excede) y lo separa en decenas/unidades con resta repetida de 10. El periférico de 7 segmentos (`pochoco_periph.v`) ya decodifica un byte "nibble alto = dígito izquierdo, nibble bajo = dígito derecho" a los patrones de segmentos vía la función `hex2seg`; para reutilizarla sin instrucción de shift, el nibble de decenas se coloca en la posición alta sumando 16 tantas veces como decenas haya (equivalente a "decenas << 4" pero hecho con `addi t0, t0, 16` en un bucle):

```

tens_div: # separa decenas de unidades (resta repetida de 10)
blt a5, t0, tens_div_done
sub a5, a5, t0
addi t1, t1, 1 # t1 = decenas
jal x0, tens_div
tens_div_done:
...
mull6: # decenas*16 sin shift (t0 = decenas*16)
beq t2, t1, mull6_done
addi t0, t0, 16
addi t2, t2, 1
jal x0, mull6
mull6_done:
add t0, t0, a5 # t0 = decenas*16 + unidades
sw t0, 0(s0) # escribe ambos digitos al periferico de 7-seg

```

Nota: el Espino Core implementa RV32E, pero las instrucciones de corrimiento (SLL/SRL/SRA/SLLI/SRLI/SRAI) están deshabilitadas en la ALU física para ahorrar LUTs en la FPGA objetivo (iCE40 HX1K) y se ejecutan como ADD si se usan por error. Por eso todo `game.s` evita corrimientos y multiplicación/división, reemplazándolas por las sumas/restas repetidas mostradas arriba.

2.5 Manejo de pulsación incorrecta

Ante un botón que no coincide con la máscara objetivo (`a4`), el programa apaga los LEDs, escribe `0xEE` en los displays ("E E" vía el decodificador hexadecimal existente), espera una pausa visible de 5 décimas de segundo ($2\,500\,000 \times 5$ ciclos, mismo esquema de resta repetida) y salta de nuevo a `round_start`. Como `a0` (`correct_count`) y `a1` (`sum_decimas`) no se tocan en esta ruta, sólo se reinicia la ronda actual, tal como exige el enunciado.

3. Assembler propio: codificación, etiquetas, validación y pruebas

El assembler (sw/assembler.py, ~510 líneas) está escrito en Python 3 puro, sin dependencias externas y sin invocar ningún assembler de RISC-V externo en ningún punto de su ejecución: toda la codificación de instrucciones a palabras de 32 bits se implementa a mano siguiendo el formato RV32I/E estándar. Recibe un archivo .s y produce un .hex con una palabra de 32 bits en hexadecimal minúscula por línea, formato consumido directamente por \$readmemh en pochoco_ram.v.

```
$ python3 sw/assembler.py sw/game.s sw/game.hex
# (sin salida = compilacion limpia; los warnings, si los hay, van a stderr)
```

3.1 Sintaxis aceptada

- Comentarios: todo lo que sigue a # en una línea se descarta.
- Etiquetas: identificador que empieza con letra, '_', '.' o '\$' seguido de letras/dígitos/'_'/'.'/'\$', terminado en ':'. Pueden ir varias etiquetas en la misma línea (a: b: addi x1, x0, 1) y pueden compartir línea con una instrucción.
- Directivas soportadas: .text, .section .text, .global/.globl — no ocupan espacio en la imagen; cualquier otra directiva (.data, .word, .align, ...) se rechaza explícitamente en vez de ignorarse en silencio, porque cambiaría la salida sin que el usuario lo note.
- Registros: por número x0..x15 o por nombre ABI (zero, ra, sp, gp, tp, t0-t2, s0/fp, s1, a0-a5). Nombres ABI de registros que existen en RV32I pero no en RV32E (a6, a7, s2-s11, t3-t6) se reconocen para poder emitir un error indicando a qué x corresponden, en vez de un "instrucción desconocida" confuso.
- Inmediatos: decimales, hexadecimales (0x...), binarios (0b...) u octales con signo opcional.
- Memoria: offset(registro), con offset opcional (por defecto 0), usado por lw/lb/lh/lbu/lhu/sw/sb/sh/jalr.

3.2 Instrucciones soportadas (usadas por el juego)

Formato	Mnemónicos	Operandos	Codificación (campos)
R	add, sub, sll, slt, sltu, xor, srl, sra, or, and	rd, rs1, rs2	funct7 rs2 rs1 funct3 rd opcode(0x33)
I (aritm.)	addi, slti, sltiu, xori, ori, andi	rd, rs1, imm	imm[11:0] rs1 funct3 rd opcode(0x13)
SHIFT	slli, srli, srai	rd, rs1, shamt	Igual a I, imm = funct7<<5 shamt (0..31)
LOAD	lb, lh, lw, lbu, lhu	rd, imm(rs1)	Igual a I, opcode 0x03
S	sb, sh, sw	rs2, imm(rs1)	imm[11:5] rs2 rs1 funct3 imm[4:0] opcode(0x23)
B	beq, bne, blt, bge, bltu, bgeu	rs1, rs2, etiqueta	imm[12 10:5] rs2 rs1 funct3 imm[4:1 11] opcode(0x63)
U	lui, auipc	rd, imm20	imm20 rd opcode(0x37/0x17)

Formato	Mnemónicos	Operandos	Codificación (campos)
J	jal	rd, etiqueta	imm[20 10:1 11 19:12] rd opcode(0x6f)
JALR	jalr	rd, imm(rs1)	Igual a I, opcode 0x67

Todas las instrucciones usadas por `game.s` pertenecen a esta lista; no se usan corrimientos porque la ALU los ejecuta como ADD (ver §2.4).

3.3 Tratamiento de etiquetas

El assembler resuelve etiquetas en dos pasadas. La primera (`find_labels`) recorre el código llevando una dirección que arranca en 0 y avanza 4 bytes por cada instrucción (las etiquetas y directivas no ocupan espacio), registrando la dirección de la instrucción que sigue a cada etiqueta; ahí mismo se detectan etiquetas duplicadas y directivas inválidas. La segunda pasada (`assemble`) codifica cada instrucción, y para `beq/bne/blt/bge/bltu/bgeu/jal` calcula el offset como `label_offset = labels[etiqueta] - direccion_de_esta_instruccion`, lo que permite referencias tanto hacia adelante (la etiqueta aún no se ha visto en el código fuente, pero ya se conoce su dirección gracias a la primera pasada) como hacia atrás. Una etiqueta usada pero nunca definida produce un error explícito en vez de una dirección basura.

3.4 Validación y mensajes de error

Cada error se reporta como un diagnóstico con archivo, línea, columna y el fragmento de código subrayado (estilo compilador), vía la clase `AssemblerError`. El assembler detecta y rechaza, entre otros:

- Mnemónico desconocido.
- Número de operandos incorrecto para el formato de la instrucción.
- Registro fuera de rango para RV32E (x16 en adelante, o nombres ABI como `s2/a6` que en RV32I existen pero no en RV32E).
- Inmediato que no cabe en el campo con signo/sin signo correspondiente (por ejemplo, un inmediato de `addi` fuera de `[-2048, 2047]`).
- Offset de salto/branch impar, o fuera del rango alcanzable (± 4 KiB para branches, ± 1 MiB para `jal`).
- Etiqueta duplicada o indefinida.
- Directiva no soportada (por ejemplo `.data` o `.word`), para no ignorar en silencio algo que cambiaría la imagen de memoria.

Además, usar una instrucción de corrimiento (`sll/srl/sra/slli/srli/srai`) no es un error — son válidas en RV32E — pero genera un `AssemblerWarning` recordando que en este núcleo se ejecutan como ADD, ya que la ALU las tiene deshabilitadas en hardware.

3.5 Ejemplo de traducción: assembly → código máquina

Tomando dos líneas reales de `game.s` (dirección 0x00 en adelante, con `s0=x8`, `a4=x15`):

Dirección	Assembly	Campos codificados	Palabra (hex)
0x00000000	<code>lui s0, 0x80000</code>	<code>imm20=0x80000, rd=8, opcode=0x37</code>	80000437
0x00000064	<code>sw a4, 4(s0)</code>	<code>imm=4-imm_hi=0,imm_lo=4; rs2=15(a4), rs1=8(s0), funct3=2, opcode=0x23</code>	00F42223

Estas dos palabras son, byte a byte, las que aparecen en `sw/game.hex` en las direcciones `0x00` y `0x64` respectivamente (verificado en las pruebas de regresión, §3.6).

3.6 Pruebas

Se agregó una batería de pruebas automatizadas en `sw/test_assembler.py`, usando únicamente `unittest` de la librería estándar de Python (sin dependencias externas, sin tocar ningún assembler de RISC-V). Se ejecuta con:

```
$ cd sw && python3 -m unittest test_assembler.py -v
```

Las 23 pruebas cubren cuatro frentes, y las 23 pasan (OK) contra la versión actual de `assembler.py`:

- Codificación por formato (9 pruebas): para R, I (positivo y negativo/complemento a 2), S, B (con reconstrucción del offset disperso en bits 31/25-30/8-11/7 para confirmar el round-trip), U y J, comparando la palabra producida contra el valor calculado a mano con la fórmula del formato, además de casos borde como offset de branch impar.
- Parseo de operandos (5 pruebas): registros por nombre ABI y por xN, rechazo de registros fuera de RV32E, inmediatos en decimal/hex/binario, y operandos de memoria `offset(registro)`.
- Validación / manejo de errores (7 pruebas): instrucción desconocida, inmediato fuera de rango, etiqueta duplicada, etiqueta indefinida, registro fuera de RV32E, directiva no soportada, y que un corrimiento (`slli`) produzca un warning y no un error.
- Etiquetas y branches (1 prueba): un programa con una etiqueta hacia adelante y otra hacia atrás, verificando que ambos offsets calculados coincidan con la distancia real en bytes.
- Regresión de extremo a extremo (1 prueba): ensambla el `sw/game.s` real y compara, línea por línea, el resultado contra el `sw/game.hex` committeado — es la misma verificación usada para confirmar que el binario cargado en la FPGA corresponde exactamente al assembly documentado en este informe, y la que se debe repetir cada vez que se modifique una constante del juego y se regenere `game.hex`.

```
Ran 23 tests in 0.002s
```

```
OK
```